



# HACKTHEBOX



## Armed Terminal

25<sup>th</sup> June 2024

Prepared By: [w3th4nds](#)

Challenge Author(s): [Leeky](#)

Difficulty: [Easy](#)

### Description:

- While venturing forth a password terminal for a door pops up. It seems simple besides the "ARMED" sign with skulls next to it.

### Objective:

- Linux binary, Anti-Static-Analysis, ARM

### Difficulty:

- [easy](#)

### Challenge:

The program consists of startup code that registers an Illegal Instruction Handler an small code snippets that end with a "undefined instruction" call.

```
00010054 void _start() __noreturn
00010054 fc029fe5 ldr r0, pointerToILLHandler {sub_10240}
00010058 ad0000eb bl registerHandler
0001005c f000f0e7 udf #0
```

The Illegal Instruction handler itself reads an index from the "undefined instructions" that raised it, and reads the next address to continue at from a table.

It also sets the Thumb flag depending on whether the address from the table is odd or even.

```

00010240 void ILLHandler()

00010240 {
00010244     int32_t ILLOpcode = *(uint32_t*)registerContext;
00010254     int32_t index = (((ILLOpcode >> 4) & 0xf0) | (ILLOpcode & 0xf));
00010260     if ((ILLOpcode & 0xf0) != 0xf0)
00010260     {
00010260         index = (ILLOpcode & 0xff);
00010260     }
00010268     void* addr = jumpTable[index];
00010270     registerContext = (addr & 0xfffffffffe);
00010288     flags = (((addr << 5) & 0x20) | (flags & 0xffffffffdf));
00010240 }

```

By following the code flow dynamically `qemu-arm -g 1234 ./armedterminal` + `gdb-multiarch -ex="target remote localhost:1234"` or by replacing the `udf` instructions with their resolved jumps we can get a clear picture:

There first is a pure ARM Mode part:

```

void _startChallenge() __noreturn
{
    if (((_readByte() == 'H' && _readByte() == 'T') && _readByte() == 'B') && _readByte() == '{'))
    {
        char r0_3 = _readByte();
        int32_t r0_4 = _readByte();
        if (((((_readByte() << 0x10) & 0xff0000) | (((uint32_t)r0_3) | (_readByte() << 0x18)) | ((uint32_t)((int16_t)(r0_4 << 8)))) ^ 0xbaadf00d) == 0xc0c08239)
        {
            int32_t r0_13 = _readByte();
            int32_t r0_14 = _readByte();
            int32_t r0_15 = _readByte();
            if (((((r0_14 << 0x10) & 0xff0000) | (((uint32_t)_readByte()) | (r0_13 << 0x18)) | ((uint32_t)((int16_t)(r0_15 << 8)))) ^ 0xf000baaa) == 0xaf34f4ee)
            {
                trap(3); // Thumb Checks
            }
        }
    }
    trap(4); // Fail
}

```

and following that a pure Thumb Mode part:

```

void _thumbCode() __noreturn
{
    if (((((((uint32_t)(_readByteThumb() + 0x2a)) == 0x89 && ((uint32_t)(_readByteThumb() - 0x4e)) == 6) && ((uint32_t)((int8_t)!(_readByteThumb())) == 0x97)
    {
        trap(2); // Correct
    }
    trap(4); // Fail
}

```

For each individual check only a very simple transformation (XOR, ADD, SUB, NEG) is done on the input and then it is compared against a constant.

From this we can manually reconstruct the flag.

```

$ qemu-arm ./armedterminal
HTB{XXX}
Correct!

```